NFPL Access Control Service Design Note

Concise submission design document - implementation-aligned

Purpose and model

This backend service makes an explicit, server-side decision about a user, an action and a shipment job. A decision combines current user status, effective role assignments, client, branch, direction, mode, workflow stage and separation of duties. Permission assignments are additive: every non-empty scope field in one assignment must match the job, while separate assignments may grant different responsibilities. The normal workflow is Confirmed -> Prepared -> Checked -> Filed; Reopened returns work to preparation.

PostgreSQL stores users, status history, jobs, role definitions, role-action mappings, scoped assignments and audit events. The pure `canPerform` policy function returns an allow/deny result, a stable code and a plain-language reason. Routes use that same function before a mutation. A mutation locks the job, checks access again, updates the job and writes its audit event in one transaction. Unknown actions, missing data, inactive users, unmatched scope and invalid states are denied.

Decisions for the 14 questions

1. Visibility versus action

`view_job` is a separate action. An internal user needs an effective in-scope view grant even if they can manage permissions. A visible job does not imply prepare, check or file access.

2. Different responsibilities in different business areas

Use several scoped assignments rather than one global role. For example, Priya is Maker everywhere and Checker only for BluePeak; Rahul is Checker only for Aster Imports, Mumbai, IMPORT. Scope dimensions in one row are ANDed.

3. A Maker who is also qualified to Check

The job records `prepared_by_user_id`. `check_job` is denied when that ID equals the actor, even if the actor has an otherwise valid Checker assignment. A different in-scope Checker is required.

4. The responsible person is absent

An absent user cannot act and receives an empty actionable queue. A supervisor can give an already-qualified substitute the smallest needed scoped grant with an expiry and a reason; this is auditable and does not copy all of the absent person's access.

5. Responsibilities change mid-process

Assignments have effective-from, expiry and revocation times, and every decision reads the current database state. A revocation affects the next check. Retained assignment rows and `user_status_history` support later investigation.

6. Cancel or reopen a Filed job

Routine cancellation is only for pre-filed work. Filed cancellation and reopening a Filed or Cancelled job require the separate `workflow_admin` role, an in-scope view grant, and a meaningful reason. The change and audit event are one transaction. This prototype has no second approver.

7. A client employee follows shipments

A `client` user receives only `client_viewer`, scoped to their own client. The policy enforces that client boundary and read-only access; other client jobs and workflow actions are denied.

8. An admin fixes a stuck job without self-escalation

`access_admin` manages roles and assignments but has no automatic shipment access. `workflow_admin` handles defined exceptions. Administrators cannot create, revoke or broaden their own grant; custom roles cannot receive permission administration or filed-job override actions.

9. Overnight automated parser

The parser is a `service` identity with a scoped Parser role. It can create a Confirmed job within scope and update only `externalReference`, `description` and `etaDate` on non-finalized jobs. It cannot change workflow or actor fields; parser operations are audited.

10. Why one person can act and another cannot

`GET /authorization/check` returns `allowed`, a stable decision code and an explanation from the same policy used for enforcement. The user can inspect their own decision; a global access administrator can inspect another user's decision. Inaccessible and missing jobs receive the same minimal 404 response.

11. Who could have changed a job at a past time

History shows who actually changed it. The investigation endpoint reconstructs the job snapshot, applies assignments effective at the requested time, and uses the recorded status effective then. Missing historical status denies eligibility rather than guessing.

12. Add a role while engineering is busy

An access administrator may create a role from approved existing actions and then grant it with scope. Role/action mappings are data-driven. The action catalogue, workflow rules, permission management and high-risk exceptions remain code-controlled.

13. Jobs waiting for me

PostgreSQL filters the queue by current status, effective grants, scope, view access, workflow stage and maker/checker separation; it counts all matches and paginates the returned page. It does not load the whole queue into Node.

14. The system is unsure

It fails closed. Missing user/job, unknown action, expired or revoked grant, invalid scope, inactive status, self-check and invalid workflow state deny access. A database failure rolls back the workflow update and its audit event.

Trade-offs and limits

Node.js, Express, PostgreSQL and `pg` keep the implementation small, explicit and explainable. The service is backend-only; no frontend is required for the demonstrated API. `x-demo-user-id` is a reproducible demo switch, not authentication and must be replaced by verified SSO/session/JWT identity in production. Audit rows are not protected from a privileged direct database writer. Production should add real authentication, audit immutability controls, dual approval for high-impact exceptions, monitoring, retention and repeatable load testing. The recorded 500-user/200,000-job check is a local observation, not a capacity guarantee.